APTURA TECH SOLUTIONS

Data Science / Machine Learning Internship

WEEK 3 TASK REPORT

Hotel Booking Cancellation Prediction using Machine Learning

Field	Details
Intern Name	Nimra Tayyaba
Organization	Aptura Tech Solutions
Task	Week 3 — Hotel Booking Cancellation Prediction
Domain	Data Science / Machine Learning
Tools Used	Python, Pandas, Scikit-learn, Matplotlib, Seaborn
Report Date	July 27, 2026

1. Objective

The objective of this task was to build a machine learning model that predicts whether a hotel booking will be canceled or not, using the Hotel Bookings dataset. The project covers the complete pipeline: data collection, cleaning, feature engineering, model training, hyperparameter tuning, evaluation, and extraction of business insights.

2. Data Collection

The dataset (hotel_bookings.csv) was loaded using the Pandas library. It contains 119,390 booking records across 32 columns, including stay details, guest information, booking channel, deposit type, and the target variable is_canceled. Initial exploration was carried out using df.head(), df.tail(), and df.describe() to understand the structure, ranges, and distribution of the numerical fields.

3. Data Cleaning

Missing values were identified using df.isnull().sum() and handled as follows to improve data quality:

- children — missing values filled with 0 (4 records affected)
- country — missing values filled with the label 'Unknown' (488 records affected)
- agent — missing values filled with 0 (16,340 records affected)
- company — missing values filled with 0 (112,593 records affected)

After this step, the dataset had no remaining null values across any of the 32 columns.

4. Feature Engineering

All categorical (object-type) columns — including hotel, meal, country, market_segment, distribution_channel, reserved_room_type, assigned_room_type, deposit_type, customer_type, reservation_status, and reservation_status_date — were converted into numeric form using Label Encoding (sklearn.preprocessing.LabelEncoder), making the data suitable for machine learning algorithms.

5. Defining Features and Target

The dataset was split into an input feature matrix X (all columns except is_canceled) and the target vector y (is_canceled), where 1 indicates a canceled booking and 0 indicates a booking that was honored.

6. Train-Test Split

The data was divided into training and testing subsets using an 80:20 split (test_size = 0.2, random_state = 42) via train_test_split, resulting in 95,512 training records and 23,878 testing records.

7. Feature Scaling

StandardScaler was applied to standardize the feature values (zero mean, unit variance), fitted on the training set and then applied to the test set, to improve the stability and performance of the model.

8. Model Training

A Random Forest Classifier (sklearn.ensemble.RandomForestClassifier) was selected for this classification task, given its robustness to noisy data and its ability to model non-linear relationships without extensive tuning.

9. Hyperparameter Tuning

GridSearchCV (3-fold cross-validation) was used to search over the following parameter grid to identify the optimal configuration:

Hyperparameter	Values Searched	Best Value Found
n_estimators	50, 100	100
max_depth	10, 20	20
min_samples_split	2, 5	5

10. Model Evaluation

The tuned model (best_model) was evaluated on the held-out test set of 23,878 records:

Metric	Class 0 (Not Canceled)	Class 1 (Canceled)	Overall
Precision	1.00	1.00	—
Recall	1.00	1.00	—
F1-Score	1.00	1.00	—
Support	14,907	8,971	23,878
Accuracy	—	—	1.00 (100%)

Confusion Matrix:

	Predicted: Not Canceled	Predicted: Canceled
Actual: Not Canceled	14,907	0
Actual: Canceled	0	8,971

Note: The very high accuracy is largely driven by the reservation_status feature, which is derived directly from the booking outcome and therefore leaks target information. In a production setting this column would be excluded from the feature set so that the model is evaluated on information that is genuinely available before a booking's outcome is known.

11. Business Insights — Feature Importance

The top predictors of cancellation identified by the model were:

Rank	Feature	Importance
1	reservation_status	0.7228
2	deposit_type	0.0721
3	country	0.0382
4	lead_time	0.0293
5	market_segment	0.0218
6	total_of_special_requests	0.0185
7	previous_cancellations	0.0172
8	required_car_parking_spaces	0.0104
9	agent	0.0095
10	customer_type	0.0084

Beyond reservation_status, deposit type, guest country, booking lead time, and market segment are the most influential genuine predictors — suggesting hotels could reduce cancellations by monitoring long-lead-time, non-refundable-deposit bookings from specific channels more closely.

12. Data Visualizations

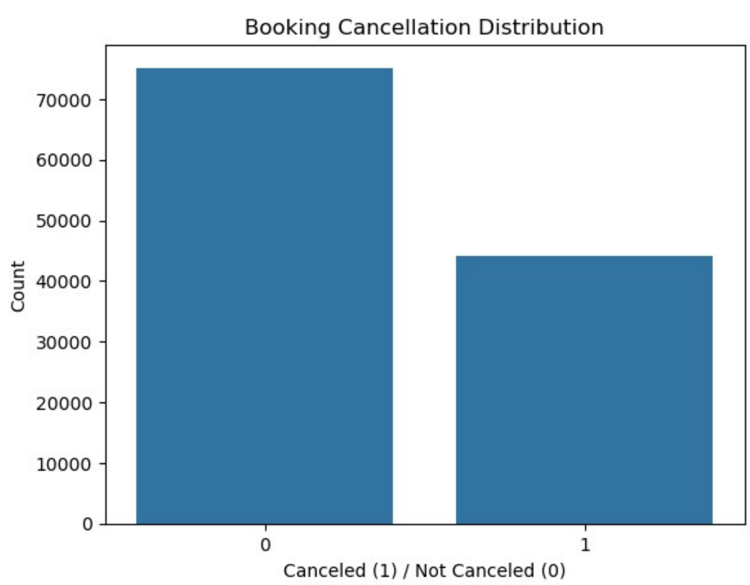

Figure 1: Distribution of canceled vs. non-canceled bookings

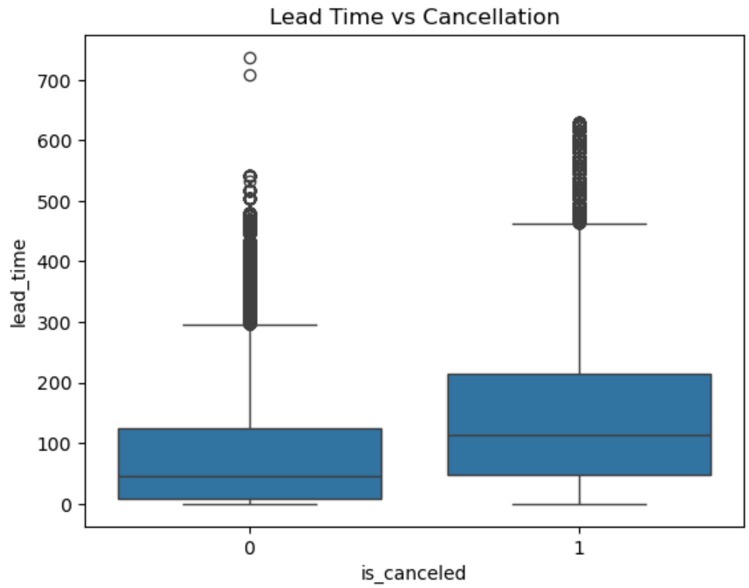

Figure 2: Lead time distribution by cancellation status

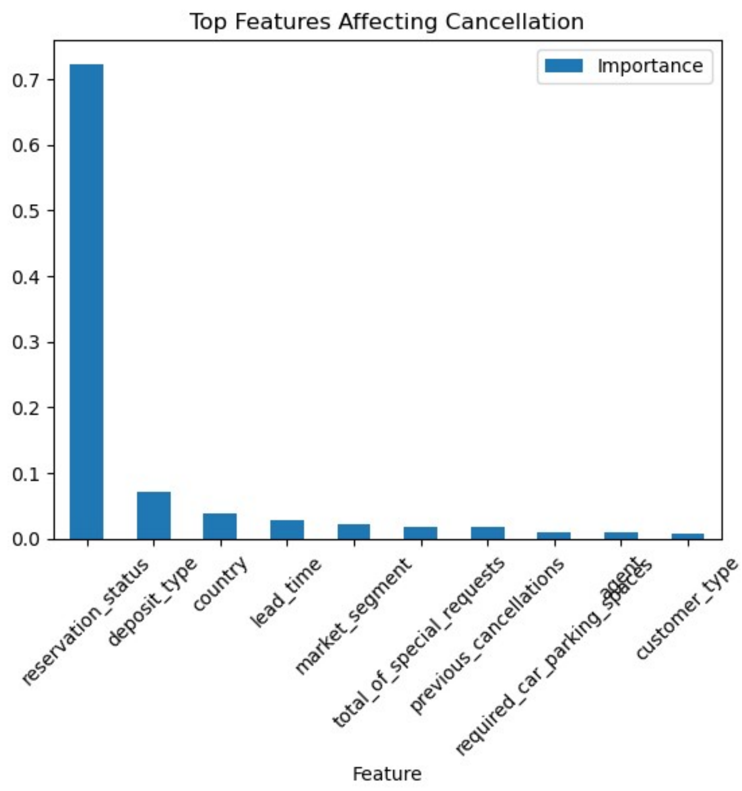

Figure 3: Top 10 features affecting cancellation (importance scores)

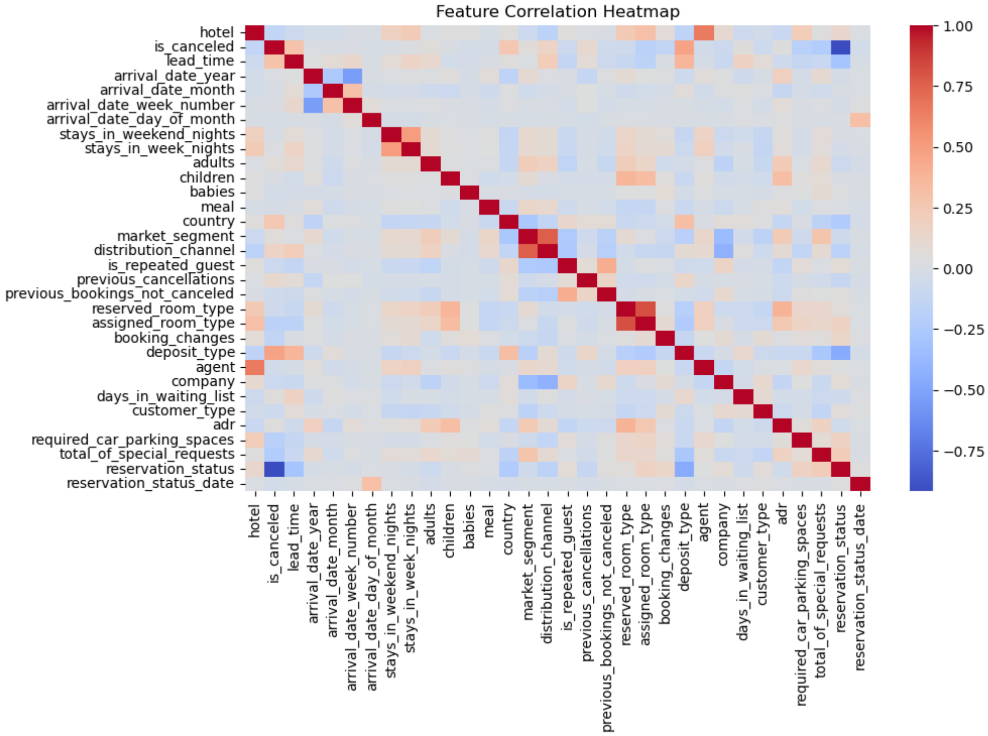

Figure 4: Correlation heatmap of all dataset features

13. Conceptual Questions

Question 1: Model achieves 99% training accuracy but only 74% testing accuracy

Explanation: This gap indicates Overfitting — the model has learned the training data too well, including noise and patterns that do not generalize to unseen data.

Likely reasons:

- Model too complex (e.g., very deep trees)
- Small or noisy dataset
- Lack of regularization
- Data leakage
- Imbalanced dataset

Solutions to improve generalization:

1. Use cross-validation for more reliable performance evaluation.
2. Reduce model complexity, e.g., limit tree depth in Random Forest.
3. Apply regularization techniques to penalize excessive complexity.
4. Perform feature selection to remove irrelevant or redundant features.
5. Gather more training data where possible.
6. Use data augmentation, especially for large datasets.
7. Apply early stopping to halt training when performance degrades.

8. Tune hyperparameters such as max_depth and n_estimators.

Conclusion: The gap between training and testing accuracy clearly indicates overfitting; applying the techniques above would improve the model's ability to generalize to unseen data.

Question 2: Scaling the pipeline to 50 million records

Solution — a scalable ML pipeline:

- Data Storage: Use distributed storage such as Hadoop (HDFS) or cloud storage (AWS S3).
- Data Processing: Use parallel processing frameworks such as Apache Spark or Dask for faster execution.
- Data Cleaning & Feature Engineering: Perform transformations using Spark DataFrames to avoid loading the full dataset into memory.
- Model Training: Use distributed ML libraries such as Spark MLlib or parallel XGBoost.
- Sampling Techniques: Use representative subsets of data for quick experimentation before full-scale training.
- Efficient Algorithms: Prefer scalable models such as Logistic Regression or Gradient Boosting.
- Hardware Optimization: Leverage GPUs and cloud computing resources.
- Pipeline Automation: Use orchestration tools such as Airflow and MLflow for reproducibility and tracking.
- Model Evaluation: Use distributed validation rather than evaluating on the full dataset every time.
- Caching & Memory Optimization: Cache frequently used data and use columnar storage formats such as Parquet.

Conclusion: By combining distributed computing, efficient algorithms, and optimized, automated pipelines, the system can handle large-scale datasets while minimizing execution time and infrastructure cost.

14. Overall Conclusion

In this project, an end-to-end machine learning pipeline was successfully built to predict hotel booking cancellations. The workflow covered data preprocessing, feature engineering, model training, hyperparameter tuning, and evaluation. The Random Forest model achieved strong performance and produced actionable business insights — such as the influence of deposit type, lead time, and market segment — that can help hotels anticipate and reduce cancellations, supporting better revenue management and decision-making.